

♦ Synchronous (Blocking)

- Tasks run **one after another**.
- The next task **waits** until the current one finishes.
- It's like standing in a queue: you cannot move forward until the person ahead of you finishes.

✓ Example (Synchronous execution in Java):

java

Copy code

```
public class SynchronousDemo {
    public static void main(String[] args) {
        System.out.println("Task 1: Start cooking...");
        cookFood(); // this must finish first
        System.out.println("Task 2: Eating food...");
        System.out.println("Task 3: Washing dishes...");
    }

    public static void cookFood() {
        try {
            Thread.sleep(2000); // simulating cooking time
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Food is cooked!");
    }
}
```

● Output (order is fixed):

arduino

Copy code

```
Task 1: Start cooking...
Food is cooked!
Task 2: Eating food...
Task 3: Washing dishes...
```

➔ Each task **waits** for the previous one.

◆ Non-Synchronous / Asynchronous (Non-blocking)

- Tasks can run **independently** without waiting for others.
- While one task is running, another can also start.
- Like cooking food in one stove while making tea in another at the same time.

✓ Example (Asynchronous using Thread):

```
java Copy code  
  
public class AsynchronousDemo {  
    public static void main(String[] args) {  
        System.out.println("Task 1: Start cooking...");  
  
        // Run cooking in a separate thread (parallel)  
        Thread cooking = new Thread(() -> {  
            try {  
                Thread.sleep(2000); // simulating cooking time  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
            System.out.println("Food is cooked!");  
        });  
        cooking.start();  
  
        // Meanwhile continue other tasks  
        System.out.println("Task 2: Eating snacks...");  
        System.out.println("Task 3: Washing dishes...");  
    }  
}
```

● Possible Output (order not fixed):

```
arduino Copy code  
  
Task 1: Start cooking...  
Task 2: Eating snacks...  
Task 3: Washing dishes...  
Food is cooked!
```

➡ Here, "Food is cooked!" appears **later**, but other tasks **did not wait**.

🎯 In Simple Words

- **Synchronous** → One by one, wait for each to finish.
- **Asynchronous** → Don't wait, tasks can run at the same time.

◆ Synchronous (Blocking)

Imagine you are **talking on a phone call** 📞

- You **say something** → then you **wait** until the other person replies.
- You **cannot move to the next line of conversation until the first one is done**.

👉 **Java synchronous = Phone Call conversation**

One task must finish before the next starts.

◆ Asynchronous (Non-blocking)

Imagine you are **chatting on WhatsApp** 

- You **send a message**, and while waiting for the reply,
- You can **do other things** → like checking Instagram, replying to another chat, etc.
- The reply will **come later**, but your work didn't stop.

 **Java asynchronous = WhatsApp Chat**

Tasks don't block each other, they can run in parallel.

 **Summary in one line:**

- **Synchronous** → Phone call (wait for response).
- **Asynchronous** → WhatsApp message (no need to wait, can do other tasks).

Java main method

In One Simple Sentence:

The main method is the starting point for any Java program. When you run a program, the Java engine looks for this specific method and begins executing your code from there.

The main Method Code:

java

 Copy

 Download

```
public static void main(String[] args) {  
    // Your code goes here  
}
```

1. public

- **Simple Meaning:** "Anyone can see this."
- **Why it's here:** The Java engine, which is outside of your program, needs to be able to find and start the main method. Making it public gives the engine permission to use it.

2. static

- **Simple Meaning:** "You don't need to create an object to use this."
- **Why it's here:** The Java engine needs to start the program before any objects (things built from your code blueprint) exist. The static keyword allows the main method to run on its own, without needing an object.

3. void

- **Simple Meaning:** "This method doesn't give anything back."
- **Why it's here:** When the main method finishes, the program ends. It doesn't need to "return" or send back a result to the Java engine. void simply means "nothing."

4. main

- **Simple Meaning:** This is the method's name.
- **Why it's here:** This is the specific name that the Java engine is programmed to look for. If you name it something else (like start), the engine won't find it, and your program won't run.

5. (String[] args)

- **Simple Meaning:** "This is a list of words someone might give the program when it starts."
- Let's break this down further:
 - String: This means "a piece of text" or "a word".
 - []: This means "a list of" or "a group of".
 - args: This is just a short name for "arguments". You can think of it as the "instructions" or "inputs" given to the program.
- **Why it's here:** It allows you to give information to your program from the outside. For example, if you had a program that edits photos, you could start it with the photo's filename as an argument: `java PhotoEditor mypicture.jpg`. Inside the program, `args[0]` would be the text "mypicture.jpg".

Simple Analogy:

Think of the main method like the **ignition in a car**.

- **public** - The ignition slot is on the dashboard where anyone (the driver) can see and access it.
- **static** - You can start the car (turn the ignition) without having to build the entire engine first.
- **void** - Turning the key starts the car; it doesn't give you back a new object.
- **main** - It's called the "ignition"—that's its specific, well-known name.
- **String[] args** - These are like the options you choose when you start the car (e.g., having the heater on or a specific radio station).

Syntax of a Static Method

```
java Copy Download  
  
accessModifier static returnType methodName(parameter1Type parameter1, parameter2  
Type parameter2, ...) {  
    // Method body: code to be executed  
    return returnValue; // (if returnType is not void)  
}
```

Breakdown in Simple Words

Let's imagine a static method is like a shared tool in a workshop. Anyone can use it without needing to own their own personal copy of the tool.

Now, let's explain each part:

1. accessModifier (e.g., public, private)

- Simple Meaning: "Who is allowed to use this tool?"
- Example: public means any other class in your program can use this method. private means only other methods *inside the same class* can use it.

2. static

- Simple Meaning: "This tool is shared. You don't need to build the whole machine (create an object) to use it."
- This is the most important keyword here. It means you can call this method directly using the class name, without creating an object first.

3. returnType

- Simple Meaning: "What kind of thing does this tool give back?"
- Examples:
 - int → gives back a number (e.g., 5)
 - String → gives back text (e.g., "Hello")
 - void → means the tool does a job but doesn't give anything back. (e.g., a tool that just prints a message).

4. methodName

- Simple Meaning: The name of the tool.
- This is what you use to call the method. It should be a verb that describes what the method does (e.g., calculateSum, printMessage).

5. (parameters)

- Simple Meaning: "What do you need to put into the tool for it to work?"
- These are the inputs for the method. You can have zero, one, or many parameters.
- Example: A `addNumbers(int a, int b)` method needs two numbers (a and b) to add them together.

6. { ... } (Method Body)

- Simple Meaning: "The instructions for what the tool does."
- This is where you write the actual code that performs the task.

7. return statement

- Simple Meaning: "This is the result, send it back!"
- This is only used if the method's `returnType` is not `void`. The value you return must match the type you promised (e.g., if you said `returnType` is `int`, you must return `10`; and not return `"hello"`);).

• How to Call a Static Method (Usage)

Because the method is static, you call it directly using the **class name**:

`ClassName.methodName(arguments);`

Example 1: Method that Returns a Value

```
java Copy Download  
  
public class MathHelper {  
    // A static method to add two numbers  
    public static int addNumbers(int firstNum, int secondNum) {  
        int result = firstNum + secondNum;  
        return result; // Sends the sum back  
    }  
  
    public static void main(String[] args) {  
        // Using the static method WITHOUT creating an object  
        int sum = MathHelper.addNumbers(5, 3); // Call it using ClassName  
        System.out.println("The sum is: " + sum); // Prints: The sum is: 8  
    }  
}
```

Example 2: Method that Returns Nothing (void)

```
java Copy Download  
  
public class Printer {  
    // A static method that does a job but returns nothing  
    public static void printWelcomeMessage(String name) {  
        System.out.println("Welcome, " + name + "!");  
        // No return statement needed because return type is 'void'  
    }  
  
    public static void main(String[] args) {  
        // Using the static method  
        Printer.printWelcomeMessage("Alice"); // Calls the method directly  
        // Output: Welcome, Alice!  
    }  
}
```

Simple Meaning of a Non-Static Method

A non-static method is a personal action or a capability of a specific object.

Think of it this way:

- A static method is like a shared utility function (e.g., `Math.sqrt()`). You just use it directly.
- A non-static method is like an action a specific person or thing can do (e.g., `myDog.bark()`, `myCar.drive()`). You need the specific person or thing (the object) to perform the action.

In one sentence: You must create an object (an instance) of the class before you can use a non-static method.

Key Differences at a Glance

Feature	Static Method	Non-Static Method (Instance Method)
Keyword	Uses static	Does NOT use the static keyword
Belongs to	The class itself	An object (instance) of the class
How to Call	ClassName.methodName()	objectName.methodName()
Can use this	No	Yes (refers to the current object)
Access Data	Can only directly access static variables	Can access both static and non-static variables

Syntax of a Non-Static Method

The syntax is almost identical to a static method, but it **omits the static keyword**.

```
java Copy Download  
  
accessModifier returnType methodName(parameter1Type parameter1, parameter2Type pa  
parameter2, ...) {  
    // Method body: code to be executed  
    // Often uses the object's data (non-static variables)  
    return returnValue; // (if returnType is not void)  
}
```

Simple Analogy: The Dog Class

Let's imagine we have a blueprint for a Dog.

- The blueprint (class Dog) has instructions for what a dog can do (methods).
- Each real dog you create from the blueprint (new Dog()) is an object (an instance).

java

 Copy

 Download

```
public class Dog {

    // A non-static variable (instance field)
    // Each Dog object gets its own copy of this data.
    public String name;

    // A non-static method (instance method)
    // This action is performed by a specific Dog object.
    public void bark() {
        System.out.println(name + " says: Woof! Woof!");
        // 'name' here refers to THIS specific dog's name
    }

    // A static method
    // This is a general utility about dogs, not a specific dog's action.
    public static void whatIsADog() {
        System.out.println("A dog is a mammal.");
    }
}
```

How to Use Them: The Key Difference

```
public class Main {
    public static void main(String[] args) {

        // --- Using the STATIC method ---
        // We can use it directly from the CLASS without any objects.
        Dog.whatIsADog(); // ✅ This works!

        // --- Using the NON-STATIC method ---
        // Dog.bark(); ❌ ERROR! Which dog? You haven't created one!

        // Step 1: First, you MUST CREATE an OBJECT (an instance)
        Dog myDog = new Dog(); // Now we have a specific dog
        Dog yourDog = new Dog(); // And here is another dog

        // Step 2: Give each object its own data
        myDog.name = "Rex";
        yourDog.name = "Fido";

        // Step 3: NOW you can call the non-static method on a SPECIFIC OBJECT
        myDog.bark(); // ✅ Output: Rex says: Woof! Woof!
        yourDog.bark(); // ✅ Output: Fido says: Woof! Woof!
```

```
// Step 1: First, you MUST CREATE an OBJECT (an instance)
Dog myDog = new Dog(); // Now we have a specific dog
Dog yourDog = new Dog(); // And here is another dog

// Step 2: Give each object its own data
myDog.name = "Rex";
yourDog.name = "Fido";

// Step 3: NOW you can call the non-static method on a SPECIFIC OBJECT
myDog.bark(); // ✅ Output: Rex says: Woof! Woof!
yourDog.bark(); // ✅ Output: Fido says: Woof! Woof!

// Each object performs the same action but on its own data.
}
```

Why Use Non-Static Methods?

They are essential for **Object-Oriented Programming (OOP)** because they define the **behavior of objects**.

They allow each object to:

1. **Have its own state** (through non-static variables like name).
2. **Perform actions** based on that state (like a dog barking its own name).

Summary

Concept	Explanation
Non-Static Method	A personal action belonging to an object .
Requirement	You must create an object with new ClassName() before using it.
Called On	An object (e.g., myObject.method()).
Purpose	To work with or modify the unique data (non-static variables) that belongs to a specific object.

That is the **golden rule** for non-static methods. You summarized it perfectly in two simple steps.

The Two Golden Steps:

1. Create an Object
2. Call the method using that object.

Simple Analogy Recap:

- **Class** = A blueprint for a Dog.
- **Non-Static Method** = An action like bark(). The blueprint describes *how* to bark.
- **Object** = A **real dog** you create from the blueprint (e.g., myDog).
- **Calling the Method** = Telling your specific, real dog to bark (myDog.bark()).

You can't tell the *blueprint* to bark. You need a *real dog* to perform the action.

Code Example Showing Your Two Steps:

```
// Step 0: The Blueprint (The Class)
public class Dog {
    // A non-static method (an action a Dog can do)
    public void bark() {
        System.out.println("Woof! Woof!");
    }
}

public class Main {
    public static void main(String[] args) {

        // --- STEP 1: CREATE AN OBJECT ---
        // Use the 'new' keyword to build a real Dog from the blueprint.
        Dog myDog = new Dog(); // myDog is now a specific object

        // --- STEP 2: CALL THE NON-STATIC METHOD USING THE OBJECT ---
        // Use the dot (.) operator on the object's name.
        myDog.bark(); // Output: Woof! Woof!

        // You can create multiple objects and each one can perform the action.
        Dog yourDog = new Dog(); // STEP 1 again for a new object
        yourDog.bark();           // STEP 2 again for the new object
    }
}
```

```

    }
}

public class Main {
    public static void main(String[] args) {

        // --- STEP 1: CREATE AN OBJECT ---
        // Use the 'new' keyword to build a real Dog from the blueprint.
        Dog myDog = new Dog(); // myDog is now a specific object

        // --- STEP 2: CALL THE NON-STATIC METHOD USING THE OBJECT ---
        // Use the dot (.) operator on the object's name.
        myDog.bark(); // Output: Woof! Woof!

        // You can create multiple objects and each one can perform the action.
        Dog yourDog = new Dog(); // STEP 1 again for a new object
        yourDog.bark();           // STEP 2 again for the new object
    }
}

```

Why These Two Steps Are Necessary:

The non-static method **bark()** might need to use data that is unique to each **Dog** object (like the dog's name, age, or breed). By requiring you to create an object first, Java ensures that the method has a specific set of data to work with.

parameterized methods.

A parameterized method is a method that can accept input values to work with. Think of it like a vending machine:

- The method is the vending machine.
- The parameters are the coin slots and button for selecting a drink (e.g., "soda", "juice").
- The arguments are the actual coin you insert and the specific button you press (e.g., a dollar bill and pressing "Coke").
- The return value is the drink you get out.

Syntax

```
accessModifier returnType methodName(DataType parameter1, DataType parameter2,
...) {
    // Method body uses the parameters to do its work
    return value; // (if not void)
}
```

Key Parts Explained:

1. (DataType parameter1, ...)

- This is the **parameter list** inside the parentheses.
- **DataType**: The kind of data the method expects (e.g., int, String, double).

- parameter1: A variable name for that input. You use this name inside the method to refer to the value that was passed in.
-

2. Arguments

- These are the **actual values** you *pass into* the method when you call it.
- They must match the order, number, and data types of the parameters.

Analogy: The Washing Machine

Imagine a wash method in a WashingMachine class.

- **Parameters** are the settings on the machine (e.g., water temperature, cycle type).
- **Arguments** are the specific choices you make (e.g., "Hot" water, "Delicate" cycle).
- The **method** uses your choices to run the wash cycle correctly.

```
// The method definition with parameters
public void wash(String temperature, String cycleType) {
    System.out.println("Washing with " + temperature + " water on the " + cycleType + " cycle.");
    // ... code to actually wash the clothes ...
}
```

Code Examples

Example 1: Simple Calculator (Returns a value)

```
public class Calculator {  
  
    // Parameterized method to add two numbers  
    // 'a' and 'b' are the parameters  
    public static int addNumbers(int a, int b) {  
        int sum = a + b;  
        return sum; // Returns the result  
    }  
  
    public static void main(String[] args) {  
        // Calling the method with arguments (5 and 3)  
        int result = addNumbers(5, 3); // 5 and 3 are the arguments  
        System.out.println("5 + 3 = " + result); // Output: 5 + 3 = 8  
  
        // You can reuse it with different arguments  
        int newResult = addNumbers(10, 20); // 10 and 20 are the new arguments  
        System.out.println("10 + 20 = " + newResult); // Output: 10 + 20 = 30  
    }  
}
```

Example 2: Greeting Generator (void method)

```
public class Greeter {  
  
    // Parameterized method to print a greeting  
    // 'name' and 'timeOfDay' are the parameters  
    public static void greetPerson(String name, String timeOfDay) {  
        System.out.println("Good " + timeOfDay + ", " + name + "! Have a great day.");  
        // No return statement needed (void)  
    }  
  
    public static void main(String[] args) {  
        // Calling the method with arguments  
        greetPerson("Alice", "morning"); // "Alice" and "morning" are the arguments  
  
        // Output: Good morning, Alice! Have a great day.  
  
        greetPerson("Bob", "evening"); // Different arguments  
        // Output: Good evening, Bob! Have a great day.  
    }  
}
```

Example 3: Non-Static Parameterized Method

```
public class BankAccount {  
    private double balance; // instance variable  
  
    // Parameterized method to deposit money  
    // 'amount' is the parameter. It works with the object's specific data ('balance')  
    public void deposit(double amount) {  
        balance = balance + amount;  
        System.out.println("Deposited: $" + amount + ". New balance: $" + balance);  
    }  
  
    public static void main(String[] args) {  
        BankAccount myAccount = new BankAccount(); // Create an object  
  
        // Call the non-static method with an argument  
        myAccount.deposit(50.0); // 50.0 is the argument for the 'amount' parameter  
  
        myAccount.deposit(25.5); // 25.5 is the argument for the 'amount' parameter  
    }  
}
```

Term	Definition	Example in the Code
Parameter	A variable in a method's definition. It's a placeholder for the incoming data.	(int a, int b)
Argument	The actual value you pass into the method when you call it.	addNumbers(5, 3)
Why Use Them?	Makes methods flexible and reusable . You write the logic once and can use it with many different values.	The same addNumbers method can add any two integers.

